Lesson — Abstraction and Behavior Design in Java

From Abstract Classes to Lambda Expressions

Learning Objectives

By the end of this lesson students should be able to:

1. Explain abstraction in OOP
2. Use abstract classes and abstract methods
3. Understand when to use interfaces
4. Implement behavior using anonymous inner classes
5. Recognize functional interfaces
6. Write behavior using lambda expressions

1. What we already have

In previous lessons we learned:

Inheritance

- class CashierStaff extends Staff

Polymorphism

- Staff staff = new CashierStaff();
- staff.can(action);

Java decides which method to execute at runtime.

2. Remaining design problem

Consider the Staff class.

Is Staff a real role?

CoffeeShop only has:

- Cashier
- Barista
- Manager

So this should not be allowed: Staff s = new Staff();

Because Staff is only a concept.

3. Abstraction

Definition

Abstraction means: defining what a system must do without specifying how it does it.

Java supports abstraction using:

- abstract class
- abstract method
- interface

4. Abstract Class

An abstract class is a class that:

- cannot be instantiated
- may contain abstract methods
- serves as a base class

Example:

```
public abstract class Staff {

    protected String name;

    public Staff(String name) {
        this.name = name;
    }

    public abstract boolean can(String action);

}
```

Now this is impossible: new Staff("John");

5. Subclasses must implement abstract methods

Example: CashierStaff

```java
public class CashierStaff extends Staff {

    public CashierStaff(String name) {
        super(name);
    }

    @Override
    public boolean can(String action) {
        return action.equals("CREATE_ORDER");
    }

}
```

Example: BaristaStaff

```java
public class CashierStaff extends Staff {

    public CashierStaff(String name) {
        super(name);
    }

    @Override
    public boolean can(String action) {
        return action.equals("CREATE_ORDER");
    }

}
```

```
@Override
public boolean can(String action) {
    return action.equals("MAKE_COFFEE");
}
```

6. Interface

Sometimes we do not want to share code. We only want to define behavior rules.

Example:

```
interface Permission {

    boolean can(String action);

}
```

A class can implement the interface:

class CashierStaff implements Permission

7. Abstract Class vs Interface

Feature	Abstract Class	Interface
Inheritance	single	multiple
Contains fields	yes	usually no
Contains method implementations	yes	limited

Typical usage:

- **Abstract class → shared structure**
- **Interface → behavior contract**

8. The next design problem

Suppose we want to temporarily check staff permissions.

Example:

- filter active staff
- sort menu items
- validate order

Creating a full class for each behavior is heavy.

Example:

```
class ActiveStaffFilter implements StaffFilter {
    public boolean test(Staff s) {
        return s.isActive();
    }
}
```

Too much code.

9. Anonymous Inner Class

An anonymous inner class is a class without a name, created at the moment of use.

Example interface:

```
interface StaffFilter {
    boolean test(Staff s);
}
```

Implementation using anonymous class:

```
StaffFilter filter = new StaffFilter() {

    @Override
    public boolean test(Staff s) {
        return s.isActive();
    }

};
```

No class name. The class exists only in that declaration class.

10. CoffeeShop example

Filter active staff:

```
for(Staff s : staffs) {
    if(filter.test(s)) {
        System.out.println(s.getName());
    }
}
```

The behavior is **defined inline**.

11. The problem with anonymous classes

Anonymous classes are still verbose.

Example:

```java
Comparator<MenuItem> comparator = new Comparator<MenuItem>() {
    public int compare(MenuItem a, MenuItem b) {
        return Double.compare(a.getPrice(), b.getPrice());
    }
};
```

This leads to a new feature in Java.

12. Functional Interface

A functional interface is an interface with exactly one abstract method.

Example:

```java
@FunctionalInterface
interface StaffFilter {
    boolean test(Staff s);
}
```

Java allows these interfaces to be implemented using lambda expressions.

13. Lambda Expression

A lambda is a shorter way to write behavior.

General syntax: (parameters) -> expression

Example: StaffFilter filter = (Staff s) -> s.isActive();

This replaces the entire anonymous class.

14. Anonymous Class → Lambda Conversion

Anonymous Class

```java
StaffFilter filter = new StaffFilter() {
    public boolean test(Staff s) {
        return s.isActive();
    }
};
```

Lambda: StaffFilter filter = s -> s.isActive(); (Much shorter).

15. Real Java usage

Lambda expressions are heavily used with Streams.

```java
staffs.stream()
      .filter(s -> s.isActive())
      .forEach(s -> System.out.println(s.getName()));
```

16. Complete concept connection

The concepts are related like this:

```
Abstraction
      ↓
Abstract Class
      ↓
Interface
      ↓
Anonymous Inner Class
      ↓
Functional Interface
      ↓
Lambda Expression
```

Meaning: Java evolved from class-based behavior to function-style behavior.

17. CoffeeShop design evolution

- **Stage 1: class Staff**
- **Stage 2: abstract class Staff**
- **Stage 3: interface StaffFilter**
- **Stage 4: anonymous class**
- **Stage 5: lambda expression**

18. Practice tasks

- Task 1: Convert Staff into an abstract class.
- Task 2: Create interface: StaffFilter
- Task 3: Use anonymous class to filter active staff.
- Task 4: Convert the anonymous class to lambda expression.

19. Evaluation Checklist

Abstraction

- **At least one abstract class** in their project.
- The abstract class contains **at least one abstract method**.
- M**ultiple subclasses** that implement that abstract method differently.

Interface

- a**t least one interface** that represents behavior in their system.
- At least a **class implement that interface**.

Anonymous Inner Class

- Implement an interface **using an anonymous inner class** at least once.
- The anonymous class is used for **temporary behavior** (not as a permanent class).
- Explain **why creating a separate class would be unnecessary in that situation**.

Functional Interface

- Create or use **a functional interface (an interface with one abstract method)**.
- Correctly mark it using: @FunctionalInterface, or explain why it qualifies as one.

Lambda Expression

- Replace **at least one anonymous inner class with a lambda expression**.
- The lambda correctly implements a functional interface.
- The lambda expression compiles and works correctly in the program.